idig4120_p2_codec

APAC (Another Predictive Audio Codec), a lossless and lossy audio codec for 16-bit PCM (predictive error encoding + Rice). quant_step 1 is lossless and bit exact, larger steps are lossy with bounded error (max step/2 LSB).

Inspiration/learning material

Two informative videos on how the FLAC codec works by kleines Filmröllchen, we tried to avoid copying everything and added some things of our own by turning FLAC into a lossy and hopefully get a similar or better compression ratio as a result (see the Evaluation section)

But what is digital audio? (The FLAC Codec #1 – (Digital) Audio and PCM) Link

But how can it fit so much music? (The FLAC Codec #2 – Lossless Audio Compression) Link

Some repos: FLAC stream_encoder.c: Link

FFMpeg Shortten (.shn) decoder (uses Rice and prediction error residuals (LPC)): Link

Flac pipeline (by FLAC author): Link

Overview

The codec does what the original plan asked for:

- standardizes input formats to int16 PCM (every WAV subtype libsndfile opens, see NOTES)
- uses predictive error encoding (FIXED, LPC or LMS), stereo pairs go mid-side first
- quantizes prediction errors, closed loop from reconstructed samples so lossy error stays bounded
- uses entropy encoding (Rice, exact per block k search)
- raw and constant block fallbacks, so the contents of a block never get bigger than its 16-bit PCM data
- optional DCT pre-shaping (JPEG-style spectral quality 1-100)
- can be run from CLI with arguments (argparse), installs as the apac command
- stores info in a header describing encoding parameters, the decoder configures itself from it
- defines its own fileformat
- evaluates compression ratio, accuracy (loss), bitrate and SNR (evaluation.py)
- tests filter out incorrect files, empty files, malformed headers etc.

Structure/Responsibilities

```
idig4120_p2_codec/
├── pyproject.toml              # src layout packaging, installs the apac command
├── README.md
├── evaluation.py               # ratio, space saving, bitrate, SNR, max error
├── conftest.py                 # makes src importable for pytest
│
├── src/audio_codec/
│   ├── config.py               # constants, enums, defaults, header struct
│   ├── main.py                 # CLI
│   ├── codec/
│   │   ├── codec.py            # the pipeline
│   │   ├── entropy.py          # Rice coding (vectorized), zigzag, best_k
│   │   └── format.py           # container spec: header and block bytes
│   ├── preprocessing/
│   │   └── pipeline.py         # wav validation
│   └── shared/
│       ├── load_file.py        # file IO, wav subtype routing
│       └── validation.py       # ValidationIssue
│
├── test/
│   ├── test_format.py          # 13 format tests
│   └── test_codec.py           # 27 codec tests
│
├── notebooks/
│   ├── codec_results.ipynb     # the results notebook: sweeps, ladders, the loader bug, edge cases
│   ├── codec_standalone.ipynb  # runs from the codec source alone, 15-file table
│   ├── fold1/                  # 15 demo clips for the standalone notebook
│   └── unseen/                 # held out wav for the standalone notebook's unseen-input section
│
└── data/
    ├── raw/fold1/              # UrbanSound8K fold1 corpus, 253 excerpts tracked
    ├── raw/unseen/             # held out wav, tracked, input for the unseen round trip test
    ├── compressed/             # scratch outputs, not tracked
    └── decompressed/           # scratch outputs, not tracked
```

(**init**.py files left out of the tree)

Who did what

Part	Who
Pipeline	Nikolai
Format specification	Nikolai
AI-declaration	Katharina
Documentation	Katharina & Nikolai
File loader	Katharina
Tests	Katharina

NOTES:

- Katharina was sick for a significant portion of the development time, so this should be considered when reading this table.
- There was communication throughout the project, so do not take the above listings as entirely representative of work distribution as both members were involved to some extent in most parts of development.

Usage

```
pip install -e .

apac --compress -i in.wav -o out.apac --quality 100    # lossless
apac --compress -i in.wav -o out.apac                  # lossy, quality 50 (default)
apac --compress -i in.wav -o out.apac --quant-step 8
apac --compress -i in.wav -o out.apac --dct-quality 75
apac --decompress -i out.apac -o back.wav
```

quality 0-100 maps onto the quantization step: 100 is lossless, every 20 points lost doubles the step (81-100 -> 1, 61-80 -> 2, 41-60 -> 4, 21-40 -> 8, 0-20 -> 16). The default 50 lands on step 4, the measured FLAC crossover point. Decompress reads everything from the header, so it has no knobs. If the Scripts dir is not on PATH use python -m audio_codec.main instead of apac.

One file metrics: python evaluation.py original.wav compressed.apac (--decoded out.wav to reuse a decoded file).

Notebooks

notebooks/codec_standalone.ipynb runs from the codec source alone and works on the shipped clips in notebooks/fold1 and notebooks/unseen, nothing else needed.

notebooks/codec_results.ipynb reads data/raw. The repo ships 253 of the 873 fold1 excerpts, enough for the demo and corpus sections. For the full-release sweep (all 10 folds, 8732 clips), set DOWNLOAD_DATASET = True at the top of the first cell: the next cell then downloads the full release from Kaggle (https://www.kaggle.com/datasets/chrisfilo/urbansound8k) into data/raw before the rest of the notebook runs. It needs pip install kagglehub, no Kaggle account, and files already present are kept, so reruns only add what is missing. With the default False the full-release sweep covers only the shipped clips and prints a note saying so.

Processing Pipeline

Encoding:

Input -> load_wav -> mid-side planes -> (DCT) -> blocks -> prediction -> quantize errors -> Rice (or raw/constant fallback) -> APAC file

Decoding:

APAC file -> header -> blocks by control byte -> Rice decoding -> inverse quantization -> reconstruction -> inverse mid-side -> trim to frames -> PCM Audio

Prediction is closed loop on reconstructed samples, that is what keeps lossy error bounded and makes lossless decode bit exact.

Our thinking

Why the pipeline looks like it does:

- Most importantly; the pipeline is quite closely related to how the FLAC pipeline is constructed, with some modifications and tweaks. Our thought was to implement a lossy version of FLAC that would allow for better compression using most of the same components. We believe this has been accomplished, showing the potential gains of adding residual quantization to FLAC as a measure to improve the compression ratio without too great a loss.

- int16 is the working format everywhere. This was done to simplify the codec and to standardize inputs. The choice of int16 was due to it being a middle ground between the high and low fidelity quanta, the decoder never touches floats, wider inputs are cast once on load.

- stereo pairs go mid-side before prediction. Correlated channels leave the side plane small, and every plane block carries its own Rice k, so side is close to free. M and S need 17 bits. A plane that does not compress when processed is stored as raw PCM16, so the worst case is the PCM size.

- prediction removes what the entropy coder would otherwise pay for. FIXED orders 1-4 are cheap, LPC fits each block, LMS adapts sample by sample. Honest result: the results notebook prices LPC across orders 1-32 and the per-block fit rarely earns its side information. The best order is 1-2 (a per-block fitted gain): a one point win over fixed-2 on the impact clip, half a point behind on the tonal clip, 10 points behind on the quiet clip, and lms-4 stays ahead of every order. From order 2 up the windowed fit degrades on two of three clips while every extra coefficient adds 2 bytes of side information per block plane. In most cases it increases the size of the compressed files.

- the quantizer runs closed loop: the predictor always works from reconstructed samples, never the originals. One decision, two guarantees: lossy error cannot drift (it stays within step/2) and lossless decode is bit exact on the same code path.

- Rice coding with an exact k search per block: we price every k with its true bit cost and take the cheapest. If Rice still costs 2 bytes a sample (noise) the block is stored raw, and a block of one constant value costs 3 bytes. The contents of a block never get bigger than its PCM data.

- the DCT pre-shaping is an encoder side setting only, the decoder is a no-op and the header records what was done. It works like the JPEG quality parameter, one scalar 1-100 with per-bin steps derived from it. Honest result: the ladder prices like lossless until quality is already ruined as an example q=90 and q=50 cost the same as transparent q=100 on the trial clip while sitting at 41.7 and 30.7 dB, and even q=10 (65.2%, 21.1 dB) loses to quant_step 8 (48.7%, 60.1 dB). As a result we consider the DCT pre-shaping to be a poor and unsuitable part of the compression pipeline.

- the 33 byte header is all the side information there is. The decoder configures itself from it, decompress takes no settings, and there are no per block lengths on the wire: Rice blocks end where decoding stops, raw blocks have a fixed size.
- what separates us from FLAC: the lossy side (quantized residuals with a bounded error guarantee) and the optional DCT pre-shaping.
- empty input and other invalid data is rejected by different checks and validations throughout the different stages of loading, preprocessing, and encoding.

Challenges during implementation

- libsndfile returns all zeros when a float wav is read as int16 on our machines, an engine idling file looked like pure silence because of it. NOTES has the story, worst bug of the project.
- Rice decoding in a per sample python loop is hopeless speed wise. The decoder is now one vectorized pass (rank select plus binary lifting in numpy), same wire format.
- the raw fallback cannot store mid or side, 17 bits do not fit int16. A raw block stores the plane's source channel instead and the decoder knows the four raw/Rice combinations that rebuild L and R.
- LPC coefficients must round trip exactly or lossless breaks, so they go on the wire as clamped Q15 ints and both sides predict with the stored values.
- a hostile header could request 2^255 samples per block, an allocation bomb. The block size field is capped at 2^20 now.
- decode is still several times slower than encode, the block walker is a python loop with numpy calls per block. Known cost, accepted for now.
- there are several gains to be made by more efficient struct packing and potentially also by tweaking configurations, but this would be reserved for a potential v2.

Format

Big endian, version 1. 33 byte header, then the data section (block major, plane minor, no per block lengths on the wire):

```
offset  size  field
0       4     magic "APAC"
4       1     version (1)
5       2     header length (33, total)
7       4     sample rate
11      8     frames
19      1     channels
20      1     predictor (NONE/FIXED/LPC/LMS)
21      1     predictor order
22      1     block size (2^n, stores n, max 20)
23      2     quantization step
25      1     dct mode (OFF/QUANTIZE)
26      2     dct window size
```

```
28      4     dct param (uint32, spectral quality 1-100)
32      1     entropy coder (RICE)
```

Every block starts with a control byte: 0 = raw int16, 1-31 = Rice with k = mode - 1, constant blocks are control byte + 2 byte value. The docstring in src/audio_codec/codec/format.py is the full spec.

Evaluation

All numbers come from the full release, all 10 folds and 8732 UrbanSound8K excerpts, APAC defaults against FLAC via libsndfile (PCM_16). The full-release sweep cell in notebooks/codec_results.ipynb records the measurement. The corpus sweep cell there is the same measurement on a 60-file fold1 sample, and the standalone notebook's 15-file table covers its shipped subset.

- lossless: APAC 57.0% of 16-bit PCM (median 840 kbit/s, 9.4 bits/sample where raw PCM is 16), FLAC 47.4% (687 kbit/s), so APAC is about 1.2x FLAC, 8732/8732 bit exact round trips
- lossy: step 4 is the first step under FLAC (46.4% of PCM, median SNR 67.6 dB, worst-file SNR 17.6 dB), steps 8/16/32 reach 40.1 / 34.0 / 28.0% of PCM (6.4 / 5.4 / 4.5 bits/sample, 566 / 475 / 393 kbit/s median)
- throughput on test-machine: encode about 16.6 MB/s, decode 7.6 MB/s

Notes:

- The loader from P1 was replaced with a more accurate loader that avoids misrepresenting 24-bit integer WAV files as int32 (problem stemming from previous loading function) additionally there was an issue related to loading of certain formats that was mitigated during development (floats being loaded all 0)

Testing

python -m pytest test

40 tests: 13 format (header and control byte round trips, every corruption path, the block size cap) and 27 codec (entropy, prediction, planes, block modes, end to end, dct quality bounds, input edges: empty wav rejected, tiny inputs, float wav routing, one held out file). conftest.py puts src on the path. The held out file test runs against the tracked wav in data/raw/unseen. Test inputs come from seeded numpy generators and the codec has no randomness, so runs are reproducible.

NOTES

Discovery of inaccuracy in libsndfile

To avoid doing too much work in terms of correct file conversion we decided to use the python package soundfile which again uses the C library libsndfile to convert between different subtypes of WAV coding. Being curious how this was done we examined the sourcecode and found that there are some scenarios where conversion does not match rounding to nearest PCM or step. An example of this is the conversion from 24-bit to 16-bit (quite relevant for us), here there is no rounding but rather a 8 bit shift keeping the 16 most

significant bits, depending on what is considered nearest (do we round up or down?) this can differ. The differences are quite negligible for our purposes, but it was interesting nonetheless.

This is why load_wav reads by WAV subtype: int16 only where libsndfile was measured correct (PCM_U8/PCM_16, ULAW/ALAW, ADPCM), and float64 with clip(round(x*32767)) for PCM_24/PCM_32/FLOAT/DOUBLE (the int16 path returns all zeros for float WAVs on this machine).

Why Rice

We chose to use Rice coding instead of standard Golomb coding to simplify implementation and leverage the performance gains from bitwise operations.